一、传统模型存在的问题

Bert类型的transformer架构中归一化层的位置对于模型大小增加和模型准确性的提升有着重要的影响

- 传统的bert类模型，随着模型大小的增加，模型性能会下降

二、传统并行方式

传统数据并行

- 通过提高batch size提高训练效率，但是过大的batch size会导致准确性下降或者收敛变慢

模型并行

Layer-wise pipeline并行：

- 一组操作在一个机器执行，并将输出传递给下一个机器执行另一组操作
- 流水线并行问题：容易遇到不一致的问题（Gpipe通过同步梯度下降解决）
    - Gpipe问题：需要额外的逻辑处理通信和计算操作的pipeline，同时会由于pipeline气泡导致效率下降

分布式tensor计算

- 可以与流水线模型并行或数据并行等同时使用

三、Background Knowledge

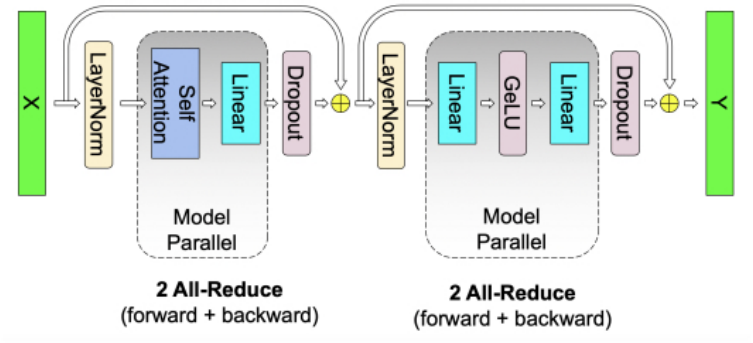
1. reduce：指将分布在不同 GPU 上的数据（通常是矩阵乘法的部分结果）进行求和或其他数学运算，并将最终结果汇聚到某一个特定的地方
2. all-reduce：它不仅会对所有 GPU 上的数据进行求和，还会将求和后的最终结果分发回所有参与计算的 GPU，这意味着操作完成后，每个 GPU 都拥有完全相同的、合并后的数据副本
3. all-gather：与 All-将分布在各个 GPU 上的不同数据部分“收集”起来，并将完整的汇总结果分发给每一个 GPU
    - All-Reduce 通常是对各部分数据进行求和等数学运算；而 All-Gather 只是简单地将各部分数据拼接在一起，使每个显卡都拥有全局的完整张量

四、Megatron-LM - 张量并行

概述

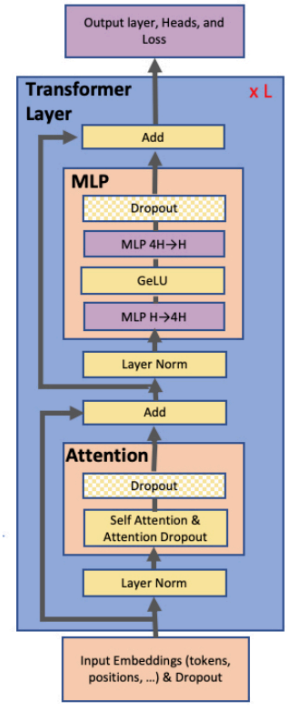
- Megatron-LM利用了Transformers架构中多头注意力计算的固有并行性，将不同的注意力头分布到不同的GPU，实现高效GPU并行
- Megatron-LM是一种简单有效的使用层内模型并行的方法，只需要使用原生pytorch，无需c++或者定制化compiler

Transformers并行



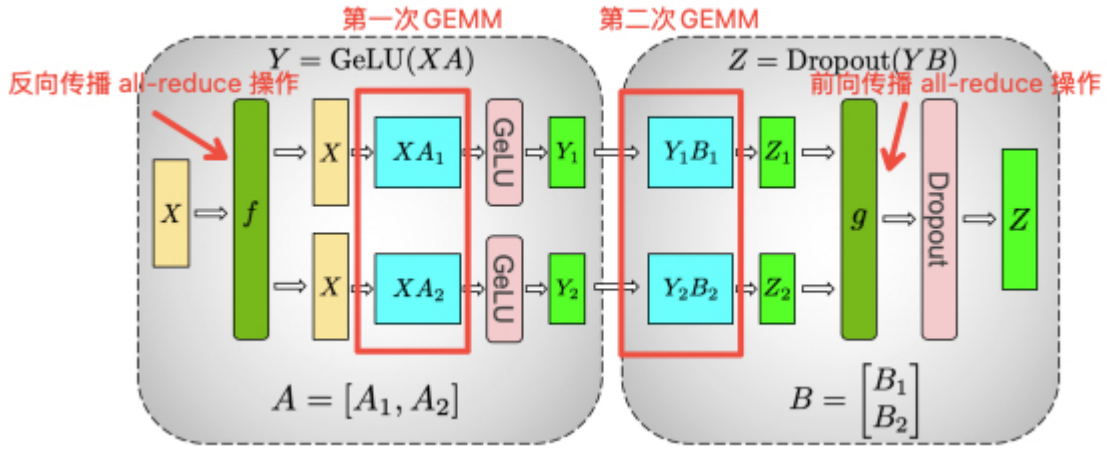
- 通过新增少量的同步原语实现一种简单的模型并行

Transformers架构



- 一个Transformers层由自注意力模块和一个两层的MLP模块组成

MLP模块

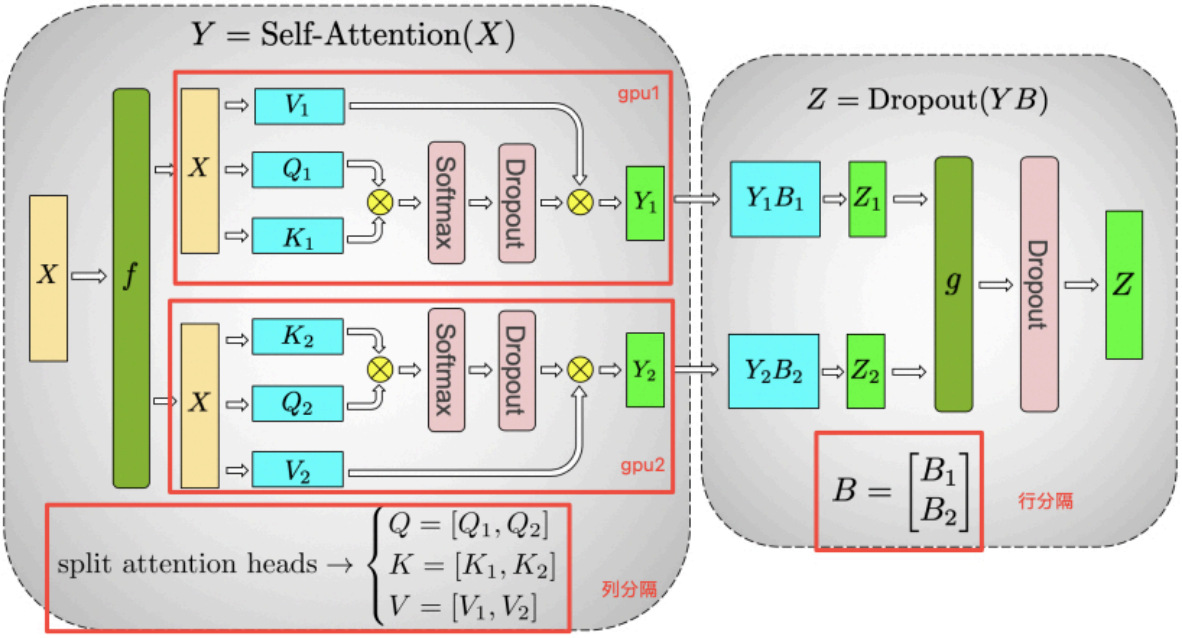


1. GEMM（XA）计算的两种并行方式：
   a. 行分隔：$X = [X_1, X_2],\ A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$
   b. 列分隔：A = [A1, A2]
2. 第一个GEMM阶段使用列分隔，第二个GEMM阶段使用行分隔
   a. 第一个GEMM使用列分隔的原因：
      i. 不使用行分隔原因： Y=GeLU(XA) 使用行分隔后 Y=GeLU(X1A1+X2A2)，但是由于GeLU为非线性函数所以 GeLU(X1A1+X2A2) != GeLU(X1A1)+GeLU(X2A2)；同时如果使用行分隔还需要在GeLU前新增一个同步节点
      ii. 使用列分隔原因：[Y1, Y2] = [GeLU(XA1), GeLU(XA2)]
   b. 第二个GEMM使用行分隔原因：
      i. 对于GeLU的输出 [Y1, Y2] 不需要任何同步通信可以直接执行
3. MLP阶段需要的通信：
   a. 前向传播需要一次 all-reduce：第二个GEMM计算后需要执行reduce后再将输出传给dropout
   b. 反向传播需要一次 all-reduce
4. f 算子（g算子类似）：

```
class f(torch.autograd.Function):
    def forward(ctx, x):
        return x   直接pass，不执行任何操作
    def backward(ctx, gradient):
        all_reduce(gradient)   同步结果
        return gradient
```

   a. 前向传播：不进行任何操作
   b. 后向传播：当梯度传回时，f 会执行 All-Reduce，为了确保在模型并行的边界上，所有 GPU 拿到的梯度是同步且一致的

Self-attention



1. 根据其内在多头机制自带的并行性，将QKV列分隔，使其每个头的工作流和参数都在一个单独的GPU，GPU间无需任何通信即可完成各自的注意力结算
2. 对于自注意力计算后的GEMM操作，执行行分隔，可以直接计算各个GPU的注意力输出，无需任何的通信

Embedding

- Transformer架构中Input Embedding和Output Embedding共享参数
- 对Embedding参数（HxV）列分隔 E = [E1, E2]
- 对于Input Embedding：在计算完成后需要一次all-reduce操作
- 对于Output Embedding：在计算完成后GEMM[Y1, Y2]=[XE1, XE2] 需要一次all-gather操作 Y = all-gather([Y1, Y2])，然后将结果送到损失函数
    - all-gather操作的通信量为BxSxV（B：batch size；S：seq len；V：词典大小），由于V一般会巨大，所以未来降低通信量，通过融合算子（GEMM[Y1, Y2]和损失函数融合），将通信量降低到BxS（因为融合后的输出为标量的损失）

核心思想：宁可重复计算，也要减少通信

- 传统做法：让一个 GPU 计算某一部分（如 Dropout 或 LayerNorm），然后将结果广播（Broadcast）给其他 GPU
- Megatron-LM 在每个 GPU 上都完整地重复运行这些轻量级的操作（Dropout、LayerNorm、残差连接）
    - 原因：这些操作的计算量相对于GEMM来说非常小，在本地重复算一遍的速度远快于通过网络进行数据同步的速度
- 实现细节：
    - 每个 GPU 都维护一份完全相同的 LayerNorm 参数副本

- 当模型并行的张量计算（即分块的矩阵乘法）完成后，每个 GPU 会立即在本地对其输出进行 Dropout 和残差连接处理，然后直接作为下一层并行区域的输入，避免额外的通信需求
- 无需同步权重更新：由于每个 GPU 上的值要么是它自己负责的本地分块（Local），要么是完全重复的副本（Duplicated），因此在训练过程中，每个 GPU 可以独立更新权重，而不需要在优化器更新阶段与其他 GPU 通信参数值